ALG-20-21-RESUELTO.pdf

JUMIACOR

Algorítmica

2º Grado en Ingeniería Informática

Escuela Técnica Superior de Ingenierías Informática y de Telecomunicación
Universidad de Granada



Reservados todos los derechos.
No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

TUS PADRES NO TE PAGARON LA CARRERA
PARA QUE SEAS EL TIPO DE PERSONA
QUE ESTUDIA POR ORDENADOR.
JD Láser IMPRENTA DIGITAL IMPRIME MÁS POR MENOS
Encuadernado gratis (mínimo 120 pág)
En B/N a 0,02€ y en Color a 0,07€

Universidad de Granada
Departamento de Ciencias de la Computación e Inteligencia Artificial
E.T.S.I. Informática y de Telecomunicación, C/Periodista Daniel Saucedo Aranda s/n - 18071 - Granada (España)

Grado en Informática

Algorítmica

Curso 2020/2021. Convocatoria ordinaria de junio
15 de junio de 2021

1. (2 puntos) Calcular el orden de eficiencia en notación $O(\cdot)$ de un algoritmo recursivo cuya expresión de tiempo es:

$$T(n) = 3T(\frac{n}{2}) + 4T(\frac{n}{4}) + n^2$$

2. (2 puntos) Dado un conjunto de n números enteros positivos $C = \{c_1, c_2, \ldots, c_n\}$ y dado un entero positivo N, se quiere encontrar un subconjunto de C, $\{y_1, \ldots, y_m\}$ que maximice $\prod_{i=1}^{m} y_i$ (se maximice el producto de los números del subconjunto), sujeto a la restricción $\prod_{i=1}^{m} y_i \leq N$. Nota: m NO es un parámetro de entrada.

   Diseñad un algoritmo de vuelta atrás para resolver este problema.

   Por ejemplo, para $n = 4$, $N = 37$ y $C = \{4, 4, 12, 3\}$, la solución es emplear el subconjunto $\{12, 3\}$ con un valor de producto igual a 36.

3. (2 puntos) Estamos en una posición (x, y) de un laberinto bidimensional representado por una matriz M de f filas y c columnas. De cada casilla se puede viajar a cualquiera de las colindantes con la casilla actual, en la misma fila o la misma columna, siempre que la casilla destino sea transitable. Los posibles valores de las casillas son $M(i, j) = t$ (transitable), $M(i, j) = n$ (no transitable) y $M(i, j) = s$ (salida). Sabiendo que disponemos del mapa del laberinto a priori, y que pueden existir varias salidas del mismo, plantee un algoritmo eficiente que devuelva el camino a la salida más próxima a la casilla inicial (x, y). NOTA: El coste de viajar de una casilla a otra es siempre 1.

4. (2 puntos) Disponemos de n tipos de monedas, las monedas de tipo i tienen un valor de $v[i]$. La cantidad de monedas disponibles de tipo i es igual a $c[i]$. Se quiere devolver una cantidad exacta M utilizando el menor número posible de monedas. Diseñad un algoritmo de programación dinámica que determine el número óptimo de monedas a usar. Aplicadlo para resolver el siguiente caso del problema, con $n = 3$, construyendo la tabla correspondiente:

$$v[\,] = (2, 4, 6),\ c[\,] = (3, 1, 2),\ M = 8$$

5. (2 puntos) Diseñad un algoritmo que determine si un cierto número natural N puede expresarse o no como producto de tres números naturales consecutivos, $Y * (Y + 1) * (Y + 2)$, con un orden de eficiencia lo mejor posible.

Duración del examen: 2 horas y 30 minutos.



WUOLAH
Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

Ej 1) $T(n) = 3T(n/2) + 4T(n/4) + n^2$

Recurrencia No Homogénea

Resolución por ecuación característica

1°) Cambio de variable $n = 2^k$ $k = \log_2(n)$

$$T(2^k) = 3T(2^k \cdot 2^{-1}) + 4T(2^k \cdot 2^{-2}) + 4^k$$

$$T(k) = 3T(k-1) + 4T(k-2) + 4^k$$

2°: Raíces

$$x^2 - 3x - 4 = (x-b)^{grado+1}$$

$$x^2 - 3x - 4 = x - 4$$

$$\frac{3 \pm \sqrt{9 - 4 \cdot 1 \cdot (-4)}}{2} \begin{cases} x = 4 \\ x = -1 \end{cases}$$

$$(x-4)^2(x+1) = 0$$

3°: Ec General

$$T(k) = (C_1 \cdot k + C_2) \cdot 4^k + C_3 \cdot (-1)^k$$

$$T(n) = (C_1 \cdot \log_2(n) + C_2) \cdot n^2 + C_3 \cdot (-1)^{\log(n)}$$

$$T(n) = C_1 \cdot \log_2(n) \cdot n^2 + C_2 \cdot n^2 + C_3 \cdot (-1)^{\log(n)}$$

$O(n^2 \cdot \log_2(n))$

Si $3^k = n^{\log_2 3}$

DUDA

$4^k \Rightarrow (n = 2^k) \stackrel{?}{=} 2^{2^k} = 2^n$? OPCIÓN A

$4^k \Rightarrow (n = 2^k) \stackrel{?}{=} 2^{k^2} = n^2$? OPCIÓN B

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

WUOLAH

Descarga carpetas completas de una vez con el Plan PRO y PRO+

CUANDO ACABA EL SHOW,
LA MAGIA CONTINÚA.

14+

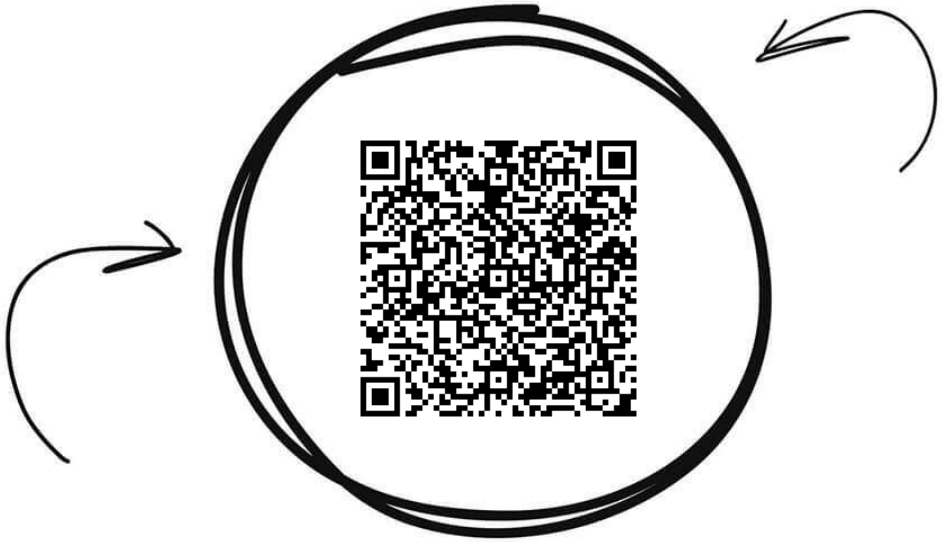


TAYLOR SWIFT
THE ERAS TOUR
(TAYLOR'S VERSION)

Ya disponible Disney+

© 2023 TAS Rights Management LLC.
© 2024 Disney y entidades relacionadas. Requiere suscripción. Se aplican términos y condiciones.

Algorítmica



Banco de apuntes de la

WUOLAH

Comparte estos flyers en tu clase y consigue más dinero y recompensas

1. Imprime esta hoja
2. Recorta por la mitad
3. Coloca en un lugar visible para que tus compis puedan escanar y acceder a apuntes
4. Llévate dinero por cada descarga de los documentos descargados a través de tu QR

1. (2 puntos) Calcular el orden de eficiencia en notación $O(\cdot)$ de un algoritmo recursivo cuya expresión de tiempo es:

$$T(n) = 3T(\frac{n}{2}) + 4T(\frac{n}{4}) + n^2$$

OTRA VEZ

$n = 2^k \quad k = \log_2(n)$

$T(2^k) = 3T(2^{k-1}) + 4T(2^{k-2}) + (2^k)^2$

$T(k) = 3T(k-1) + 4T(k-2) + 4^k$

$(x-4)$

$x^2 - 3x - 4$

$\frac{3 \pm \sqrt{9 - 4 \cdot (1) \cdot (-4)}}{2} = \frac{3 \pm 5}{2}$

$x = 4$

$x = -1$

$(x-4)^2 (x+1)$

$(C_1 \cdot k + C_2) 4^k + (C_3 \cdot 1^k$

$C_1 \cdot \log_2(n) \cdot n^2 + C_2 \cdot n^2 + C_3 \Rightarrow O(n^2 \cdot \log_2(n))$

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

WUOLAH

Descarga carpetas completas de una vez con el Plan PRO y PRO+

2. (2 puntos) Dado un conjunto de n números enteros positivos $C = \{c_1, c_2, \ldots, c_n\}$ y dado un entero positivo N, se quiere encontrar un subconjunto de C, $\{y_1, \ldots, y_m\}$ que maximice $\prod_{i=1}^{m} y_i$ (se maximice el producto de los números del subconjunto), sujeto a la restricción $\prod_{i=1}^{m} y_i \leq N$. Nota: m NO es un parámetro de entrada.

Diseñad un algoritmo de vuelta atrás para resolver este problema.

Por ejemplo, para $n = 4$, $N = 37$ y $C = \{4, 4, 12, 3\}$, la solución es emplear el subconjunto $\{12, 3\}$ con un valor de producto igual a 36.

Backtracking

REPRESENTACIÓN DEL PROBLEMA / SOLUCIÓN

La representación de la solución se hará mediante un vector "sol" que contendrá enteros componentes de la solución final.

RESTRICCIONES EXPLÍCITAS

El vector solución estará compuesto de enteros "j" tal que $j \geq 1$ y $j \leq N$.

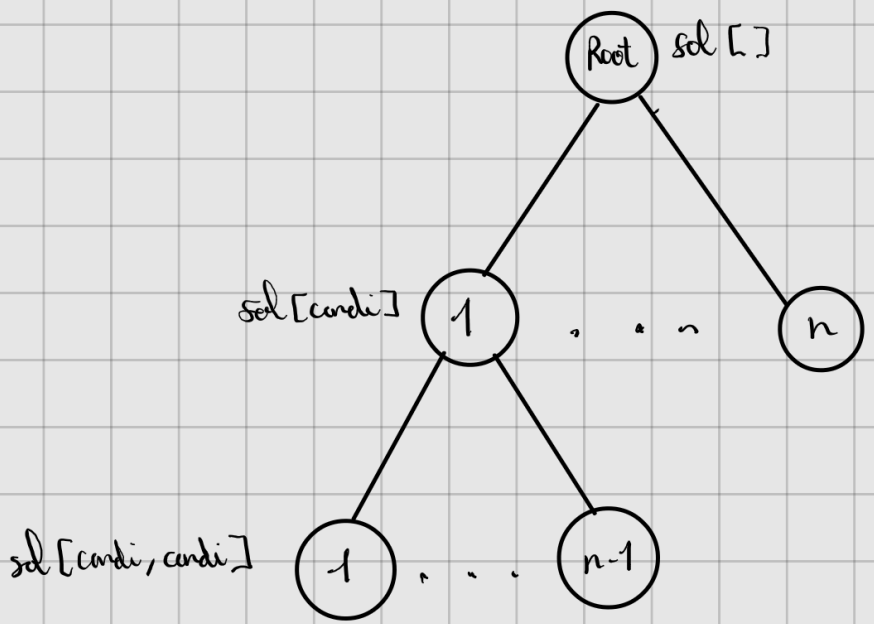
RESTRICCIONES IMPLÍCITAS

Los componentes del vector han de ser únicos (un candidato sólo se escoge una vez), y el producto de sus componentes ha de ser $\prod_{i=1}^{m} j_i \leq N$.

FUNCIÓN DE PODA

En cuanto encontremos subconjuntos cuyo producto sea mayor que N o menores que la cota global, descartaremos la futura solución resultante.

ÁRBOL IMPLÍCITO El factor de ramificación creará implícito a cada nodo, este es el número de candidatos sin escoger; la profundidad simboliza a un candidato seleccionado.



Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

WUOLAH

Descarga carpetas completas de una vez con el Plan PRO y PRO+



ALGORITMO

```
bool backProducto (vector &candi, vector &sol, int prod, int N) { // Devuelvo solución por referencia
  int mejor = 0;
  if (prod ≤ N && candi.empty()) // Función Objetivo
      if (prod > mejor) { Actualizo COTA GLOBAL
          mejor = prod;
          print "Una solución", prod, sol;
          return true;
      }
  }
  if (prod > N || candy.empty()){ // Función Poda
      return false;
  }
  candidato = candi.pop_back(); // Se prueba con el primer candidato
  if (factible (candidato, prod, N)){ // Función Factibilidad
      prod *= candidato;
      sol.push_back(candidato);
      if (backProducto (candi, sol, prod, N)) // Probamos con el
          return true;                        // candidato en sol
      }
      prod /= candidato;
      sol.pop_back(); } Rollback
  }
  }
  candi.pop_back(); // Se prueba sin el primer candidato
  if (backProducto (candi, sol, prod, N){
      return true;
  }
  return false
}
```

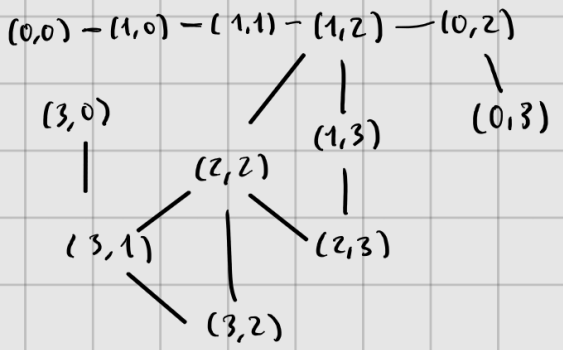
Ponte un cafeeeé con aroma del DIA
¡Compra ya!
*Dia recomienda el consumo responsable de bebidas con cafeína
WUOLAH
Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

3. (2 puntos) Estamos en una posición (x, y) de un laberinto bidimensional representado por una matriz M de f filas y c columnas. De cada casilla se puede viajar a cualquiera de las colindantes con la casilla actual, en la misma fila o la misma columna, siempre que la casilla destino sea transitable. Los posibles valores de las casillas son $M(i, j) = t$ (transitable), $M(i, j) = n$ (no transitable) y $M(i, j) = s$ (salida). Sabiendo que disponemos del mapa del laberinto a priori, y que pueden existir varias salidas del mismo, plantee un algoritmo eficiente que devuelva el camino a la salida más próxima a la casilla inicial (x, y). NOTA: El coste de viajar de una casilla a otra es siempre 1.

NO B&B
O BACKTRACKING
GREEDY !!

Grafo ponderado, dirigido.

IDENTIFICAMOS DIJKSTRA QUE AL SER PESO 1 → SERÍA ANCHURA



- Un nodo en este problema es una posición transitable (t) del laberinto (x, y)

- Candidatos - nodos pendiente de calcular el camino mínimo
- Escogidos - nodos para los que ya se calculó el camino mínimo.
- F. Selección - seleccionará el nodo de "CANDIDATOS" que tenga menor distancia al "origen".

COMO TODAS LAS ARISTAS DEL GRAFO TIENEN EL MISMO COSTE ES COMO UN RECORRIDO EN ANCHURA

- F. Factibilidad - sólo se pueden seleccionar "nodos" transitables y pertenecientes al laberinto, pero estos serán los nodos candidatos.
- F. Solución - Cuando se llegue a una casilla de tipo "s"
- F. Objetivo - Minimizar el camino a la salida

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

WUOLAH

Descarga carpetas completas de una vez con el Plan PRO y PRO+

```
set<vertices> dijkstraLaberinto (Grafo G(V,A), nodo raiz) {
    for (int i=0; i<V.size(); i++) {
        dist[vi] = INF;
        padre[vi] = null;
    }
    set<vertices> S; // seleccionados
    distancia[raiz] = 0; // distancia de raiz a raiz es 0. IMPORTANTE
    priority_queue Q;
    Q.insert(raiz); // meter la raiz
    for (int i=0; i<V.size(); i++) Q.insert(vi);
    bool salida_encontrada = false;
    while (!Q.empty && !salida_encontrada) { //Objetivo
        v = Q.top(); //Se prueba con un vértice
        Q.delete(v); //Se saca de la cola
        if (v.tipo == "t" || v.tipo == "s") { //Factibilidad
            S.insert(v);
            V' = v.verticesAdyacentes();
            for (int i=0; i<V'.size(); i++) {
                if (dist[v'(i)] > dist[v] + camino(v, v'(i))) {
                    dist[v'(i)] = d[v] + camino(v, v'(i));
                    padre[v'(i)] = v;
                }
            }
            if (v.tipo == "s") {
                salida_encontrada = true;
            }
        }
    }
    return S; //Solución => Devolvemos los seleccionados
}
```

Actualizamos distancias y padres de los vértices

importante para la p-q

Solución

* MUST

METER TODOS LOS V EN Q

* HAY QUE METER LOS VÉRTICES

Terminamos

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

WUOLAH

Descarga carpetas completas de una vez con el Plan PRO y PRO+

4. (2 puntos) Disponemos de n tipos de monedas, las monedas de tipo i tienen un valor de $v[i]$. La cantidad de monedas disponibles de tipo i es igual a $c[i]$. Se quiere devolver una cantidad exacta M utilizando el menor número posible de monedas. Diseñad un algoritmo de programación dinámica que determine el número óptimo de monedas a usar. Aplicadlo para resolver el siguiente caso del problema, con $n = 3$, construyendo la tabla correspondiente:

$$v[\,] = (2, 4, 6),\ c[\,] = (3, 1, 2),\ M = 8$$

(cantidades)

Pr. Dinámica

Monedas con cantidades limitadas

1 REPRESENTACIÓN DEL PROBLEMA

- Secuencia de decisiones: Se toma o no la moneda de tipo "i".

M[i, j] => M[n+1, M+1] (matriz con tipos de monedas como filas y cantidades como columnas)

* moneda 0;
* cantidad 0;

2 REPRESENTACIÓN DE LA OPTIMALIDAD DE BELMAN

1) Suponemos que x1...xn (monedas seleccionadas) en una solución óptima para el problema M[n, M]

2) Suponemos por contradicción que x1...xn-1 no es óptima:
   - Si xn=0; M[n-1, M]
   - Si xn=1; M[n-1, M-mn]

3) En el caso xn=0 asumo que existe una solución y1...yn-1 para M[n-1, M] con "sum_1^{n-1} yi < sum_1^{n-1} xi" y "sum_1^{n-1} yi·ci = M - xn·cn"

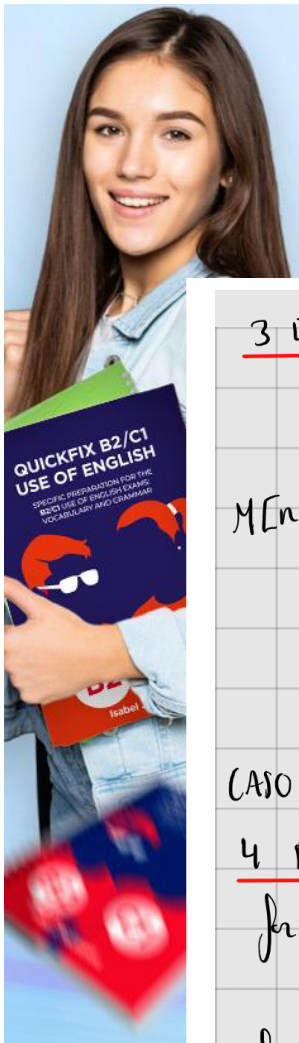
4) Pero entonces si yn = xn = 0 tenemos que: "sum_1^{n} yi·ci = M"; luego y1...yn es una solución para M[n, M], y además "sum_1^{n} yi < sum_1^{n} xi, luego no sería óptima.

Redundante

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

WUOLAH

Descarga carpetas completas de una vez con el Plan PRO y PRO+



3 DEFINICIÓN RECURSIVA

$$M[n,M]\begin{cases} M[i,j] = \left\{ \forall k \in \{0,1,\dots c[i]\} \ \min\left(M[i-1, j-k\cdot v[i]] + k\right) \right\} \end{cases}$$

c[i]

iteración

cambia la cantidad del tipo de moneda

CASO BASE : $M[0,m] = \infty$, $M[n,0] = 0$;

4 PSEUDOCÓDIGO n = v.size(); (tipos de moneda) M[n+1, M+1]

```
for (int j=0 ; j<M ; j++)
    M[0,j] = INF;
for (int i=0 ; i<N ; i++) {
    M[i,0] = 0;
```

} CASOS BASE

```
for (int i=0 ; i<N ; i++) {
    for (in j=0 ; j<M ; j++) {
        min = ∞ ;
        for (int k=0 ; k<c[i] ; k++) {
            if([M[i-1, j-k·v[i]) + k < min ]{
                min = M[i-1, j-k·v[i]] + k ;
                M[i,j] = M[i-1, j-k·v[i]] + k ;
            }
        }
    }
}
return M[n,M];
```

nº unidades de cada tipo

WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

5/6) DEVOLUCIÓN DE LA SOLUCIÓN Y MONTAJE DE TABLA

if $M[n,m] == M[n-1,m]$

// El objeto no se seleccionó, pasamos a comprobar $M[n-1,m]$

else

// El objeto sí se selecciona, pasamos a comprobar $M[n, m-v[i]]$

* CONSTRUCCIÓN DE LA TABLA

m \ n	0	1	2	3	4	5	6	7	8
0	0	∞	∞	∞	∞	∞	∞	∞	∞
2(3)	0	∞	(1)	∞	2	∞	3	∞	4
4(1)	0	∞	(1)	∞	1	∞	2	∞	3
6(2)	0	∞	(1)	∞	1	∞	1	∞	(2)
tipo(con)									

Solución {6, 2}

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

WUOLAH

Descarga carpetas completas de una vez con el Plan PRO y PRO+

5. (2 puntos) Diseñad un algoritmo que determine si un cierto número natural N puede expresarse o no como producto de tres números naturales consecutivos, $Y * (Y + 1) * (Y + 2)$, con un orden de eficiencia lo mejor posible.

DyV

$N = Y * (Y+1) * (Y+2)$; Ej $120 = 4 \cdot 5 \cdot 6$

- Candidatos => $1 \ldots \underbrace{\sqrt[3]{N}}_{\text{int}}$; si no caigo $1 \ldots N$ tb vale
- Mitad => mitad = (inicio + fin) / 2
  mitad (1 + int($\sqrt[3]{N}$)) / 2 *

DyV Ej5 (inicio (1), fin ($\sqrt[3]{N}$), N)

// Búsqueda Binaria

```
mitad = (inicio + fin) / 2;
if (N == mitad * mitad+1 * mitad+2) {
    return true;
}
else {
    if (N < mitad * mitad+1 * mitad+2)
        DyV (inicio, mitad-1);
    else
        DyV (mitad+1, fin);
}
return false;
```

$T(n/2)+1$

$O(\log_2(n))$

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

WUOLAH

Descarga carpetas completas de una vez con el Plan PRO y PRO+

5. (2 puntos) Diseñad un algoritmo que determine si un cierto número natural N puede expresarse o no como producto de tres números naturales consecutivos, $Y * (Y + 1) * (Y + 2)$, con un orden de eficiencia lo mejor posible.

N = Y * (Y+1) * (Y+2) ? ; low = 1 High = $\sqrt[3]{N}$

```
int EjSDyV ( int low, int high, int N) {
    mitad = (low + high) / 2;
    if (N == (mitad * (mitad+1) * (mitad + 2))
        return mitad
    else {
        if ( N < (mitad * (mitad+1) * (mitad + 2)))
            return EjSDyV ( low, mitad-1, N);
        else
            retur EjSDyV (mitad+1, high, N);
    }
}
```

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

WUOLAH

Descarga carpetas completas de una vez con el Plan PRO y PRO+